

Lecture 8. MSA Communications



이영곤



Microservices Communication Architecture

- ✓ synchronous/asynchronous communication scheme
- ✓ Kubernetes Network Structure
- ✓ Architecture Patterns

MSA Features

- ✓ Separating systems by service
- ✓ Independent Distribution by Service
- ✓ Network communication between services required
- ✓ Disability isolation is possible
- ✓ Technology Stack Diversity

Features of MSA Communication

- ✓ Need to collaborate between services
- ✓ Distributed System Environments
- ✓ Need to consider network failure
- ✓ Need to manage data consistency
- ✓ Scalability and Performance Considerations

synchronous communication

- ✓ REST
- ✓ gRPC

Infrastructure-based communications

- ✓ Service Mesh

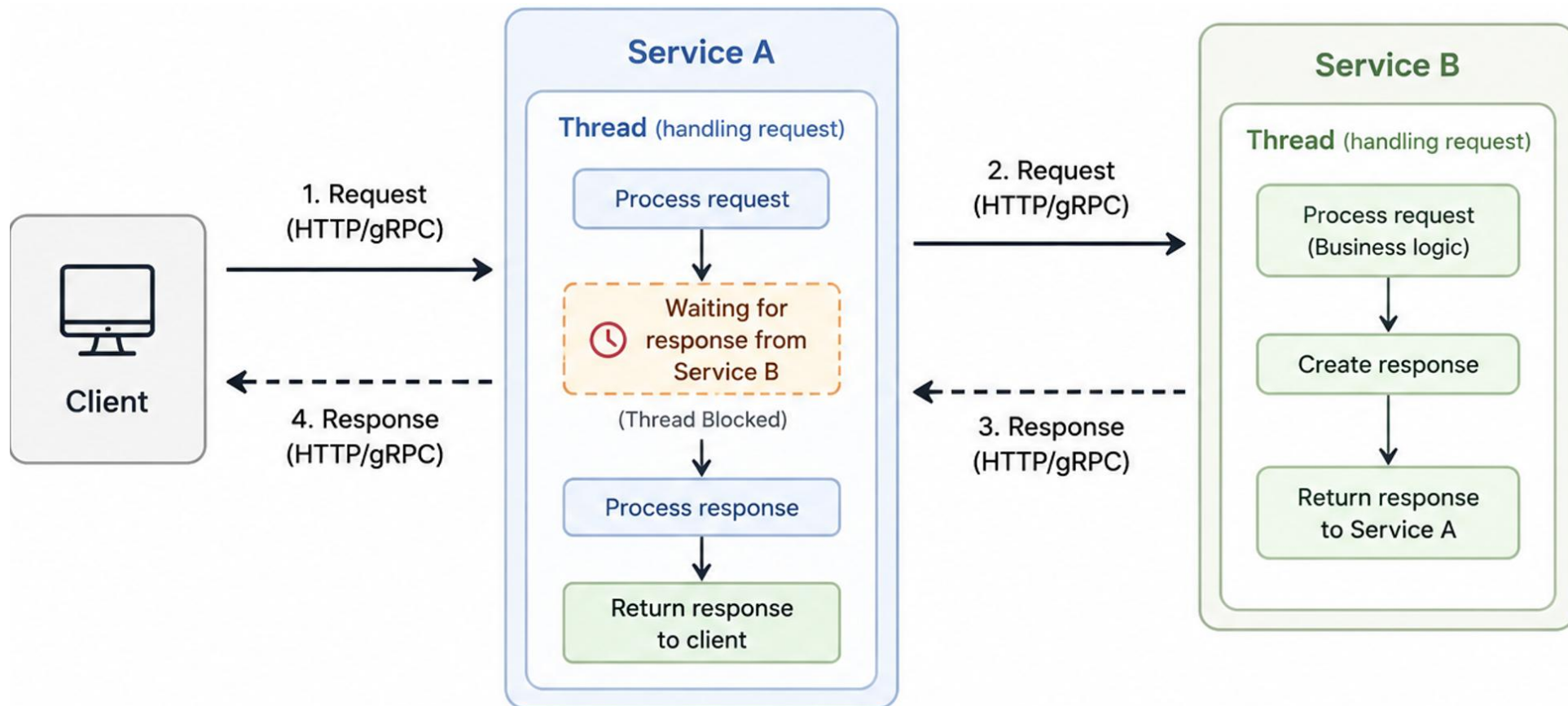
asynchronous communication

- ✓ Message Queue
- ✓ Event Streaming

synchronous communication

- ✓ wait for a response after a request
- ✓ HTTP-based communication
- ✓ Common API Call Method
- ✓ Advantages:
 - Simple to implement
 - Ready to see your response
- ✓ Disadvantages:
 - strong coupling of services

synchronous communication



REST communications

- ✓ HTTP-based API communication
 - Using JSON Data Format
 - the most widely used method
- ✓ example
 - GET /orders/100
 - POST /orders

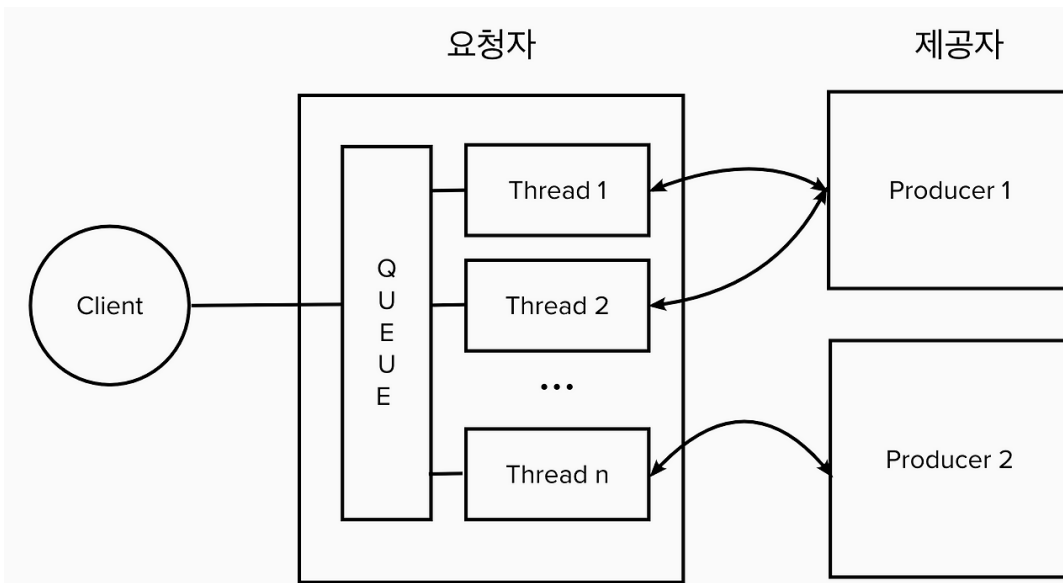
REST communications

- ✓ Advantages:
 - Simple Implementation
 - Easy to Debug
 - Support for a variety of languages
- ✓ Disadvantages:
 - Network latency
 - Service failure propagation is possible

How to implement REST communication: RestTemplate

✓ Features:

- Synchronous Method
- Expected to be deprecated after Spring 5
- Use for simple service calls



@RestController

```
public class BookController {
```

@Autowired

```
RestTemplate restTemplate;
```

@GetMapping("/call")

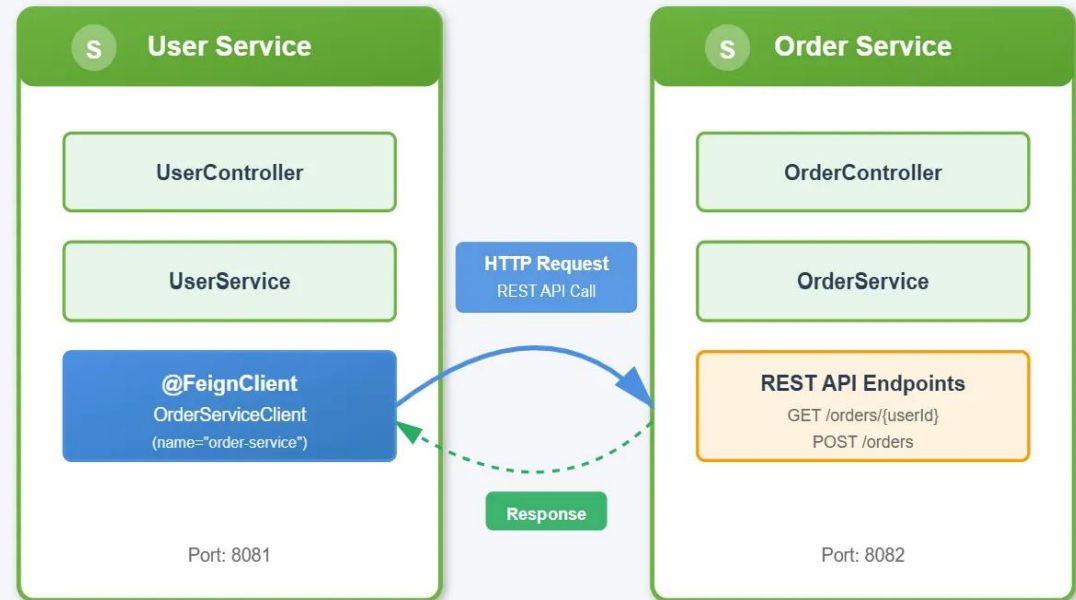
```
public String callService() {
    return restTemplate.getForObject(
        "http://book-service/books",
        String.class
    );
}
```

How to implement REST communication: OpenFeign

✓ feature:

- Interface-based REST calls
- The code is very simple
- Most Used by MSA

Spring Boot Microservices with Feign Client



Key Features:

- Declarative REST client using `@FeignClient` annotation
- Load balancing & service discovery support

How to implement REST communication: OpenFeign (most used by MSA)

```
@EnableFeignClients
@SpringBootApplication
public class Application {
}

@FeignClient(name="book-service")
public interface BookClient {

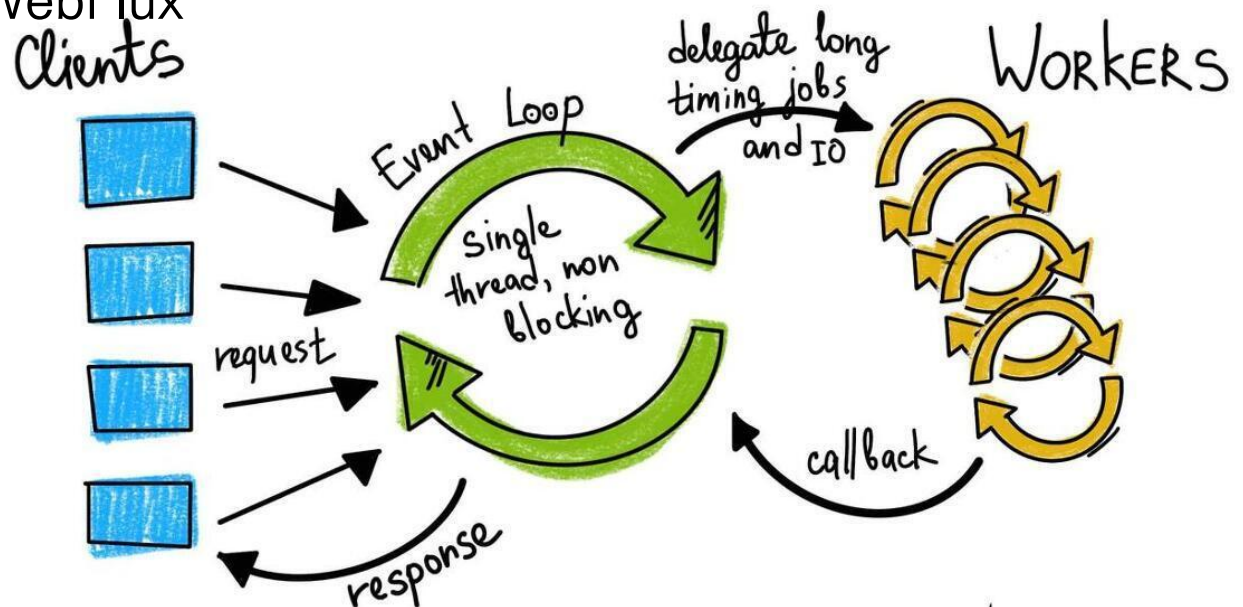
    @GetMapping("/books")
    List<Book> getBooks();
}

@Autowired
BookClient bookClient;
```

How to implement REST communication: WebClient (currently recommended method of Spring)

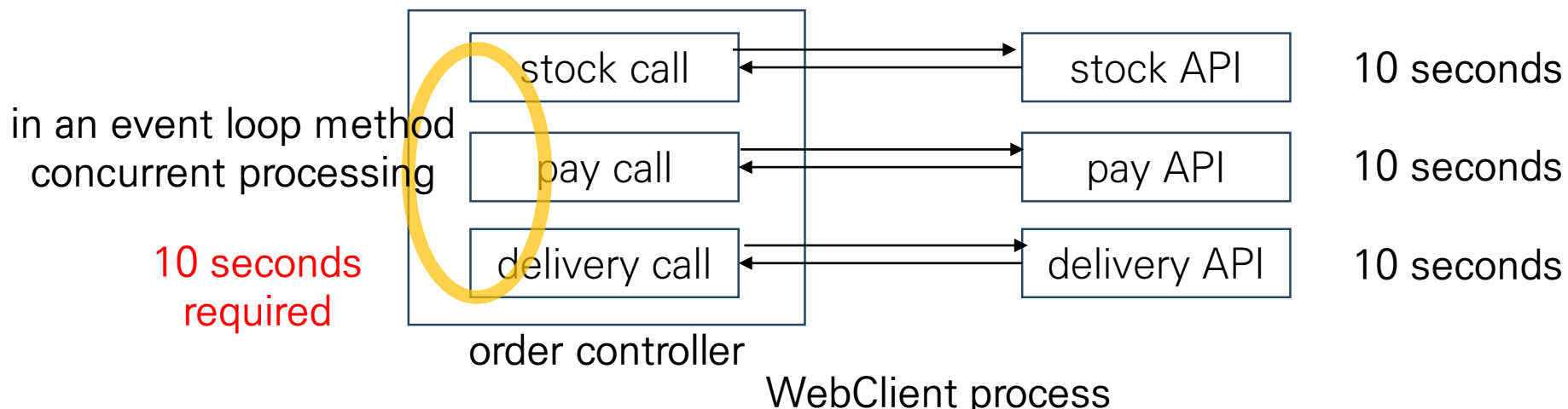
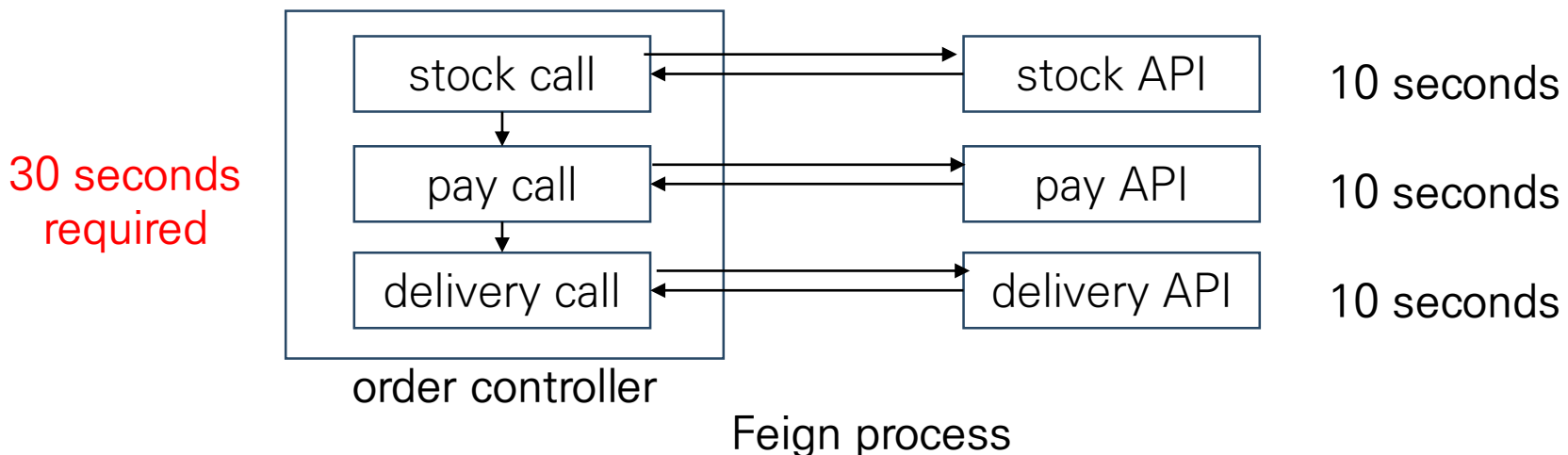
✓ feature:

- Reactive HTTP Client Appeared from Spring 5
- Asynchronous / Non-blocking
- Based on WebFlux



WebClient Practice Example

- ✓ Compare Feign and WebClient Methods



Mono and Flux Differences in the Web Client

✓ Mono

- Asynchronous processing of zero or one piece of data
- Use when results are in one case
- Example: Look up 1 user
- 1 log-in result
- Order Details 1 item

```
Mono<User> userMono = webClient.get()
    .uri("/users/1")
    .retrieve()
    .bodyToMono(User.class);
```

```
{
  "id": 1,
  "name": "Kim"
}
```

Mono and Flux Differences in the Web Client

✓ Flux

- Asynchronous processing of more than 0 pieces of data
- Use when there are multiple results
- Example: Look up a list of users
- Order List Inquiry
- Receive real-time event streams

```
Flux<User> userFlux = webClient.get()
    .uri("/users")
    .retrieve()
    .bodyToFlux(User.class);
```

```
[
  { "id": 1, "name": "Kim" },
  { "id": 2, "name": "Lee" }
]
```

WebClient Key Methods

- ✓ `Mono.zip()` = Operator to group multiple asynchronous tasks in parallel
- ✓ `block()` = asynchronous $\rightarrow$ synchronous switching (thread stop)
- ✓ extract the first value from the tuple `tuple.getT1()` = zip result

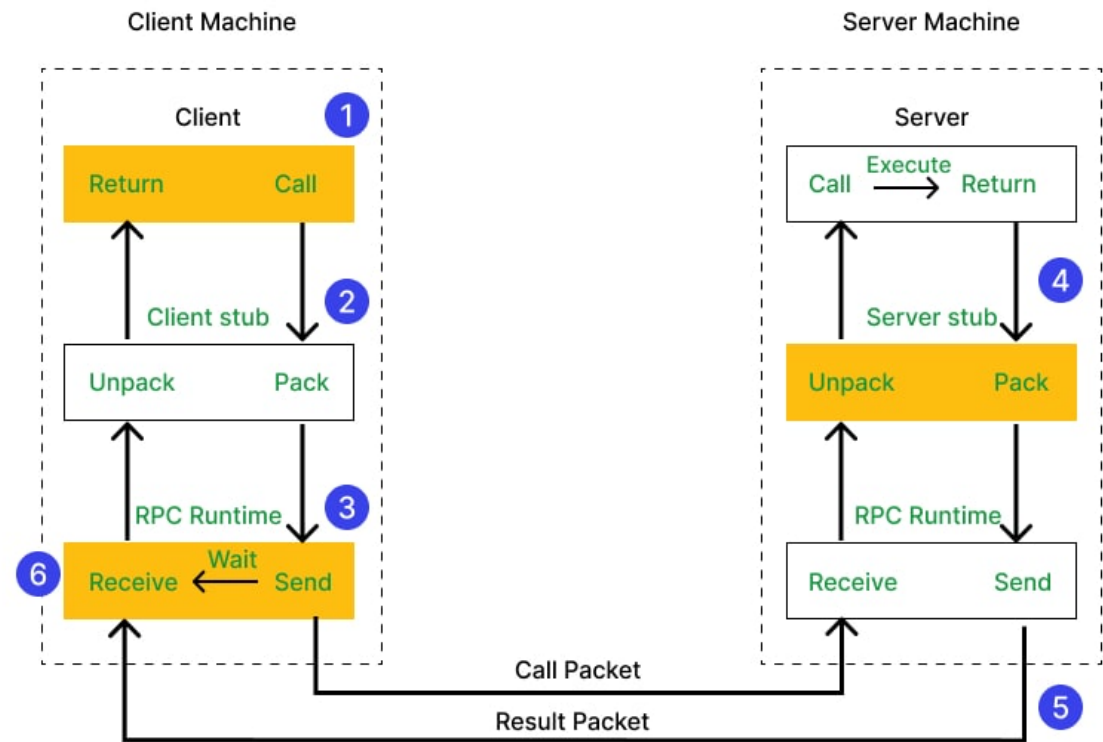
`Mono.zip(mono1, mono2, mono3)`  Run concurrently and return in bundles

`Mono<Tuple3<T1, T2, T3>>`  Result Type

mono1 
mono2 
mono3  Run at the same time $\rightarrow$ All done $\rightarrow$ Batch of results (Tuple))

gRPC communication

- ✓ RPC Framework Developed by Google
 - HTTP/2 based communication
 - Using Protocol Buffers
- ✓ Characteristics
 - high performance
 - Type stability
 - Streaming Support



REST vs gRPC comparison

✓ REST

- HTTP/1.1
- JSON
- browser-friendly

✓ gRPC

- HTTP/2
- Protobuf
- High-Performance Internal Communications

asynchronous communication

- ✓ No immediate response required after request
 - Message Broker-Based
 - Event-based architecture
- ✓ representative technology
 - Kafka
 - RabbitMQ
 - NATS

Comparison of asynchronous communication Kafka and RabbitMQ

- ✓ Apache Kafka
 - Stream large events / Log-based / Data Pipeline
- ✓ RabbitMQ
 - Traditional Message Brokers / Queue Based / Job Processing

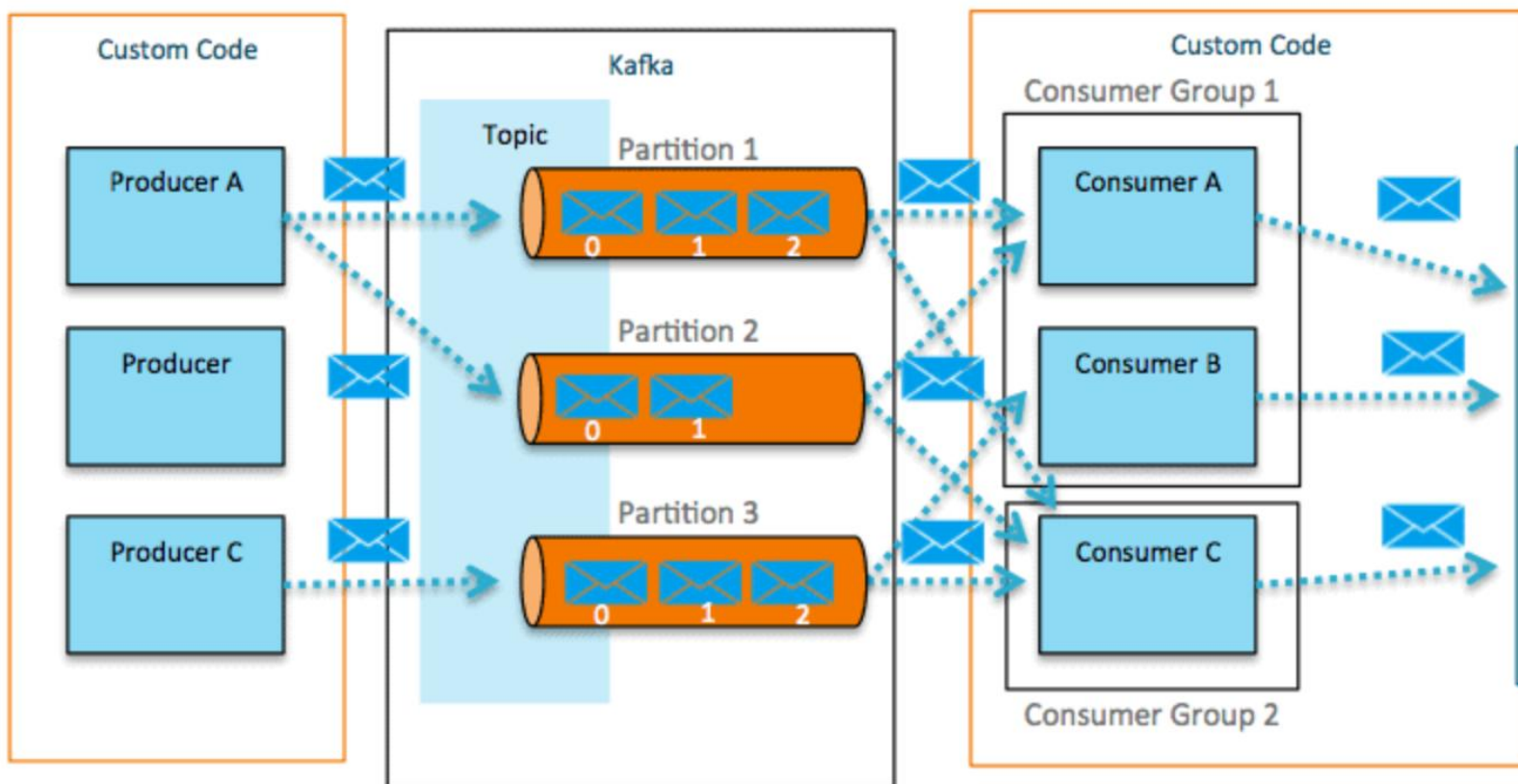
	Kafka	RabbitMQ
Model	(append-only log)	(message queue)
Save Message	Keep saving to disk (retention)	Delete when consumed
the manner of consumption	Pull (consumer takes it)	Push (delivered by broker)
Message Reuse	(Reprocessing, replay)	Basically impossible
Order guarantees	Partition-wise guarantees	Queue unit guarantees

Comparison of asynchronous communication Kafka and RabbitMQ

- ✓ Kafka (event streaming)
 - Producer → Topic (Save Log) → Consumer Group
 - Message "Not deleted and still stacked"
 - Read consumer based on offset
 - Same data read independently by multiple consumers
 - Powerful Data Pipeline for Log Analysis

Comparison of asynchronous communication Kafka and RabbitMQ

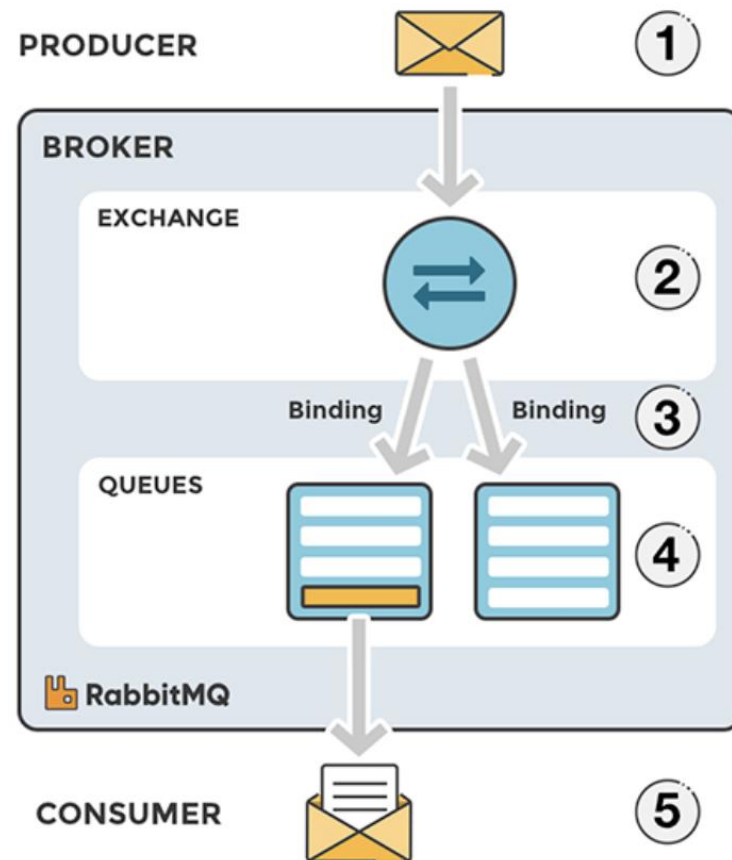
✓ Kafka (Event Streaming)



Comparison of asynchronous communication Kafka and RabbitMQ

✓ RabbitMQ (Message Queue)

- Producer → Exchange
→ Queue → Consumer
- Messages disappear when consumed
- Broker가 Consumer에게 push
- Best for work queues
- Robust for business processing, transaction



Comparison of asynchronous communication Kafka and RabbitMQ

✓ Performance and scalability

category	Kafka	RabbitMQ
Throughput	Very high (millions of TPS available)	the middle
(latency)	Slightly high	Low
Scalability	Partition-based horizontal expansion	Clusterable but limited
durability	Very Strong (Disk-Based)	Depends on settings

Comparison of asynchronous communication Kafka and RabbitMQ

✓ (Delivery Semantics)

category	Kafka	RabbitMQ
At least once	O	O
Exactly once	O (Supported)	restrictive
At most once	O	O

RabbitMQ Practice

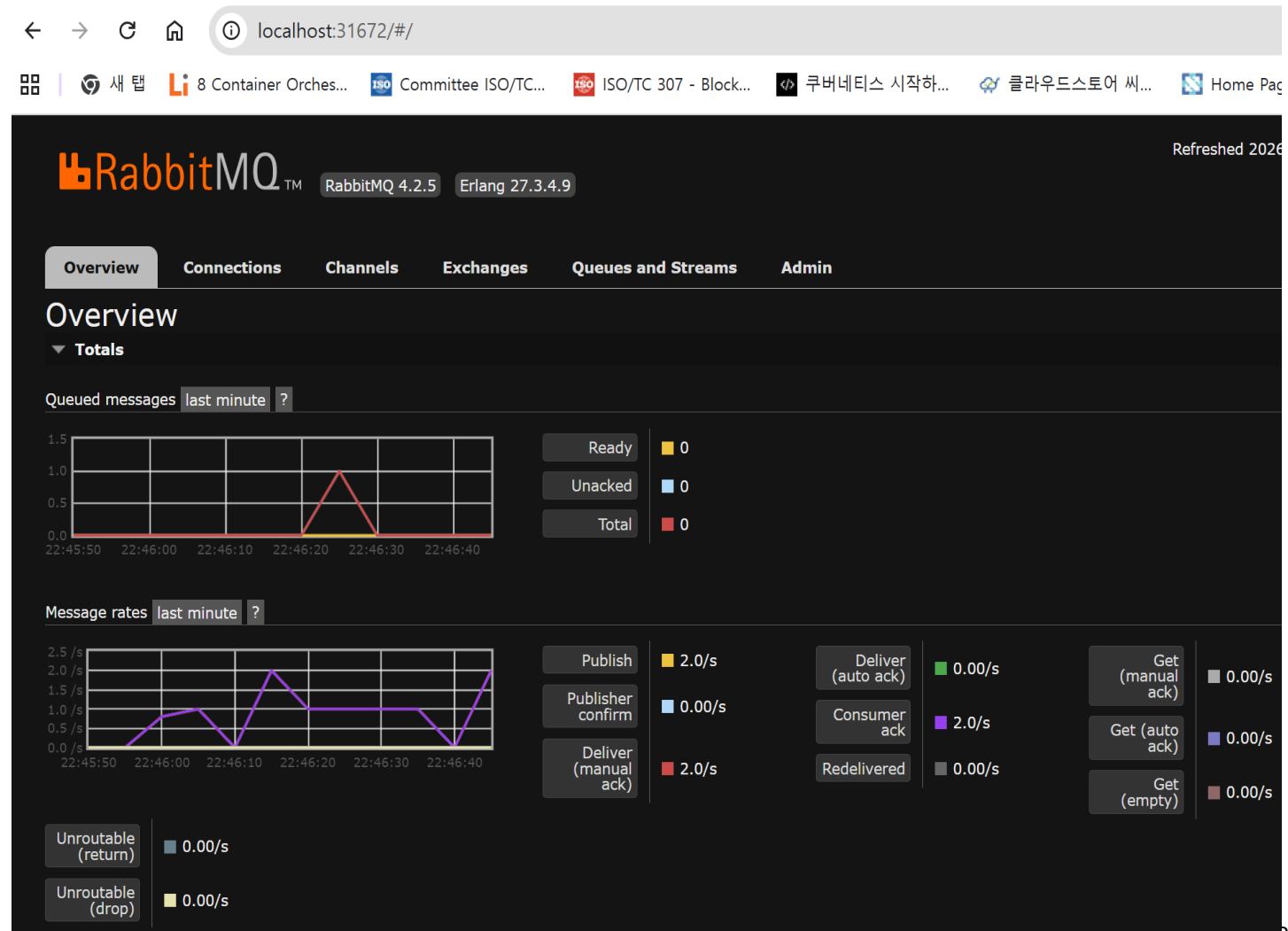
- ✓ Installing in Kubernetes
- ✓ rabbitmq.yaml use
- ✓ Check UI screen
- ✓ <http://localhost:31672> (id:admin, pwd:admin123)
- ✓ Order of execution
 - 1.exchange declaration : myexchange
 - 2.queue declaration: myqueue, key: mykey
 - 3.Create binding
 - 4.producer exchange + routing key to publish
 - 5.consume queue onconsume

RabbitMQ Practice

- ✓ With rabbitmq activated
- ✓ Run rabbitmqserver on Kubernetes:
 - `kubectl apply -f rabbitmqserver.yaml`
- ✓ Enter `http://localhost:30106/api/publisher/start` in postman method
- ✓ Run the rabbitmqclient on Kubernetes:
 - `kubectl apply -f k8s.yaml`
- ✓ Check the Web browser:

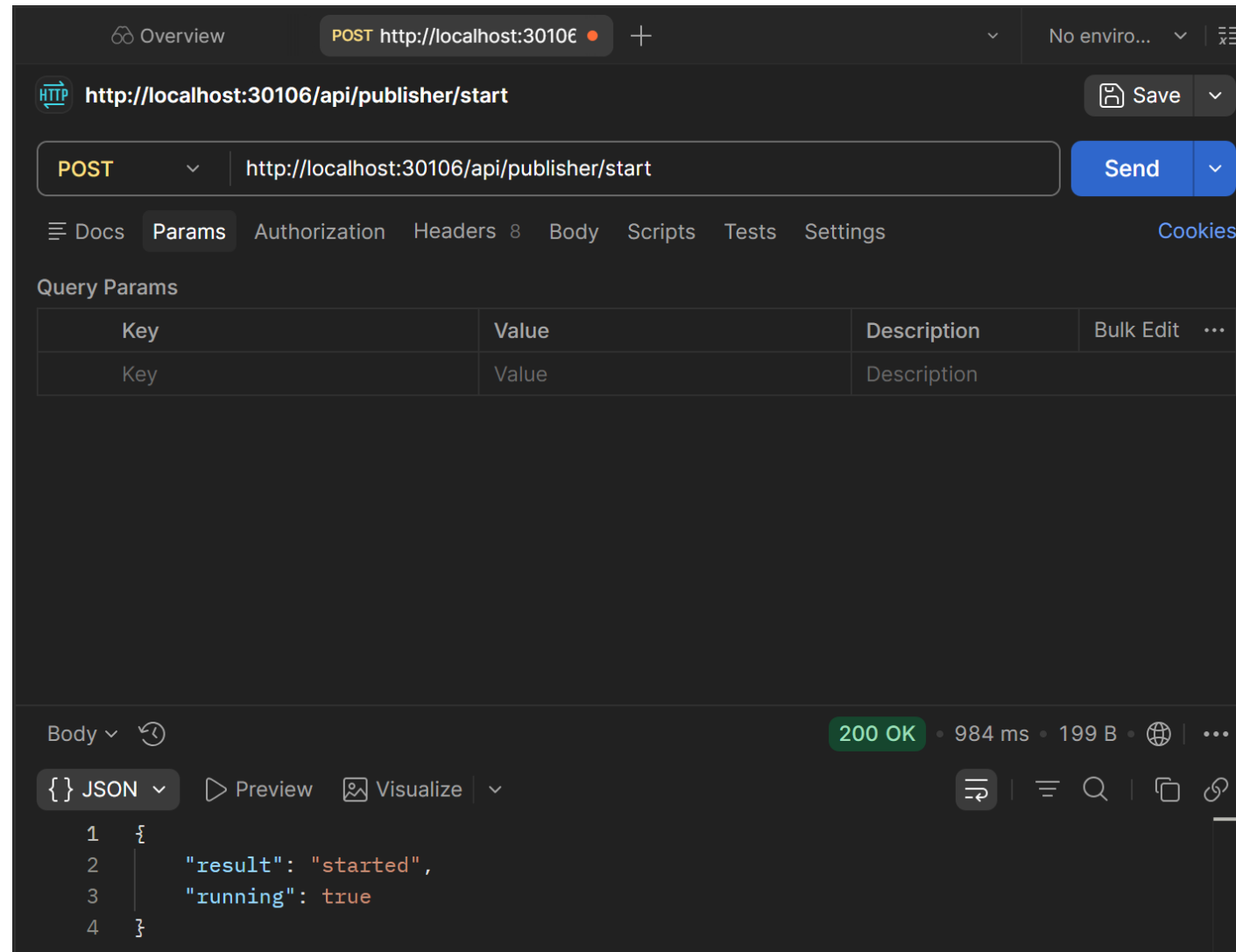
RabbitMQ Practice

- ✓ Rabbitmq operates as below



RabbitMQ Practice

- ✓ Enter as below in postman
- ✓ Create a message on the server and enter it into MQ



RabbitMQ Practice

- ✓ Check in Web browser
- ✓ Confirm receipt of messages from the client

The screenshot shows the RabbitMQ Live Feed web interface in a browser. The address bar shows 'localhost:30108'. The browser's tab bar includes several tabs, with the active one being 'Home Page - Clo...'. The main content area has a title 'RabbitMQ Live Feed' and a description in Korean: 'myexchange 에서 mykey 로 바인딩된 myqueue 메시지를 수신해 실시간으로 화면에 표시합니다.' Below this, there are four buttons: 'Exchange: myexchange', 'Routing Key: mykey', 'Queue: myqueue', and a green button labeled '실시간 연결됨'. The message feed displays three messages, each in a light gray box with a white border. The first message is 'suksoqxxacsqdtl' with a timestamp of '2026-03-19 13:51:24'. The second message is 'rwyrbubxirji' with a timestamp of '2026-03-19 13:51:23'. The third message is 'cmrrpmuhrbp' with a timestamp of '2026-03-19 13:51:22'.

localhost:30108

8 Container Orches... Committee ISO/TC... ISO/TC 307 - Block... 쿠버네티스 시작하... 클라우드스토어 씨... Home Page - Clo...

RabbitMQ Live Feed

myexchange 에서 mykey 로 바인딩된 myqueue 메시지를 수신해 실시간으로 화면에 표시합니다.

Exchange: myexchange Routing Key: mykey Queue: myqueue 실시간 연결됨

suksoqxxacsqdtl
2026-03-19 13:51:24

rwyrbubxirji
2026-03-19 13:51:23

cmrrpmuhrbp
2026-03-19 13:51:22

Disadvantages of synchronous communication

- ✓ delayed propagation
 - If A waits for B, and B waits for C,
 - When C slows down, B slows down, and eventually A slows down, too
 - latency of the downstream service propagates throughout the upstream
- ✓ disability propagation
 - If the calling destination is in a failure or timeout state,
 - Threads from called services may run out while waiting
 - This can make even normal services seem unresponsive
 - ☞ cascading failure

disadvantages of synchronous communication

- ✓ Throughput Degradation
 - Synchronization calls include wait times per request
 - The higher the TPS, the longer the thread occupancy time,
 - Eventually, overall throughput drops
- ✓ Call chain length increase problem
 - API Gateway → Order → Inventory → Payment → Notification
 - If the chain gets longer like this, the total response time is
 - ☞ Sum of delay for each stage + network overhead

If you can use the synchronous expression

- ✓ If you really need an immediate response
 - e.g. Look up prices, check inventory immediately, look up membership information on the order screen
- ✓ Synchronization calls are natural because users need to receive results now

If you prefer to switch to asynchronous (event/message)

- ✓ Post-processing that does not require immediate results
 - e.g. sending notifications, earning points, loading logs, reflecting statistics, sending mail, updating recommendation models
- ✓ These are message-based, such as Kafka/RabbitMQ, which reduces interdependence

synchronous protection on call

- ✓ Short timeout
 - Make sure you don't wait long for slow service.
 - You can also set connect timeout, read timeout in OpenFeign
- ✓ Circuit Breaker
 - Block time without continuing calls to the fault service
 - CircuitBreaker can be associated with implementations such as Resilience4j
- ✓ Isolate Bulkhead service failures from encroaching on the entire thread pool
 - Resilience4j는 Bulkhead와 ThreadPoolBulkhead를 제공

synchronous protection on call

- ✓ Retry is limited
 - It helps with temporary network errors, but reckless retries can rather amplify the failure
 - Care must be taken, especially as it has the effect of tapping harder on already slow services
- ✓ Fallback
 - If it is not a core feature, it should be able to give bypass responses such as default, cache values, and "temporarily unreachable."
- ✓ Minimize Call Chain
 - Need to reduce the structure in which a service calls another service in multiple steps again
 - Manage from request to less than 1 to 2 hops if possible

Event-based architecture

- ✓ Service A → Event Publish
- ✓ Message Broker → Event delivery
- ✓ Service B → Event Subscribe
- ✓ feature
 - Loose Coupling
 - Highly scalable
 - Eventual Consistency

Message broker role

- ✓ Delivery of messages between services
- ✓ Save Message
- ✓ Ensuring Message Order
- ✓ Asynchronous processing support

Service Discovery

- ✓ Provide Pod groups as one network endpoint
- ✓ Kubernetes provides service discovery by default
- ✓ Access Pod through Service Objects
- ✓ CoreDNS-based service name interpretation
 - 예: `http://order-service`

Kubernetes Service

- ✓ Provide Pod groups as one network endpoint
 - Provides load balancing
 - Troubleshooting Pod IP Changes
- ✓ Service Type
 - ClusterIP
 - NodePort
 - LoadBalancer

Ingress

- ✓ Forward external HTTP traffic to internal services
- ✓ URL-based routing
- ✓ TLS Termination Support
 - e.g: `/orders` → `order-service`

API Gateway

- ✓ API Management Platform
- ✓ Authentication / Authorization Management
- ✓ Rate Limit
- ✓ Logging
- ✓ API Version Management

Service mesh

- ✓ representative technology
 - Istio
 - Linkerd

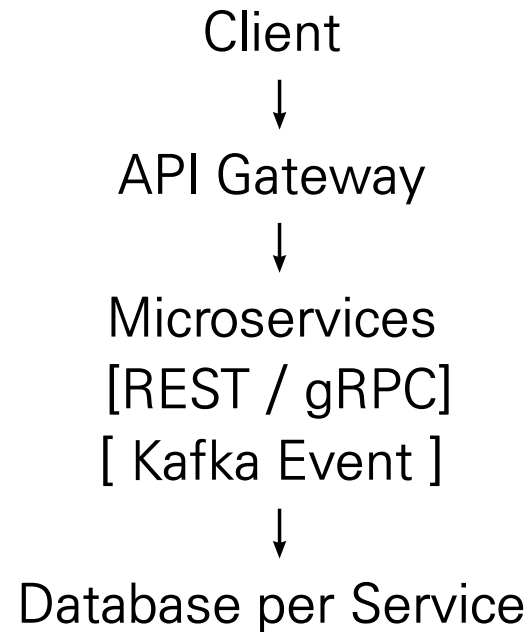
- ✓ feature
 - mTLS
 - Traffic Control
 - Observability
 - Circuit Breaker

service communication design

- ✓ Service Combination Chart
- ✓ Disability isolation
- ✓ Data Consistency
- ✓ Network latency
- ✓ Scalability

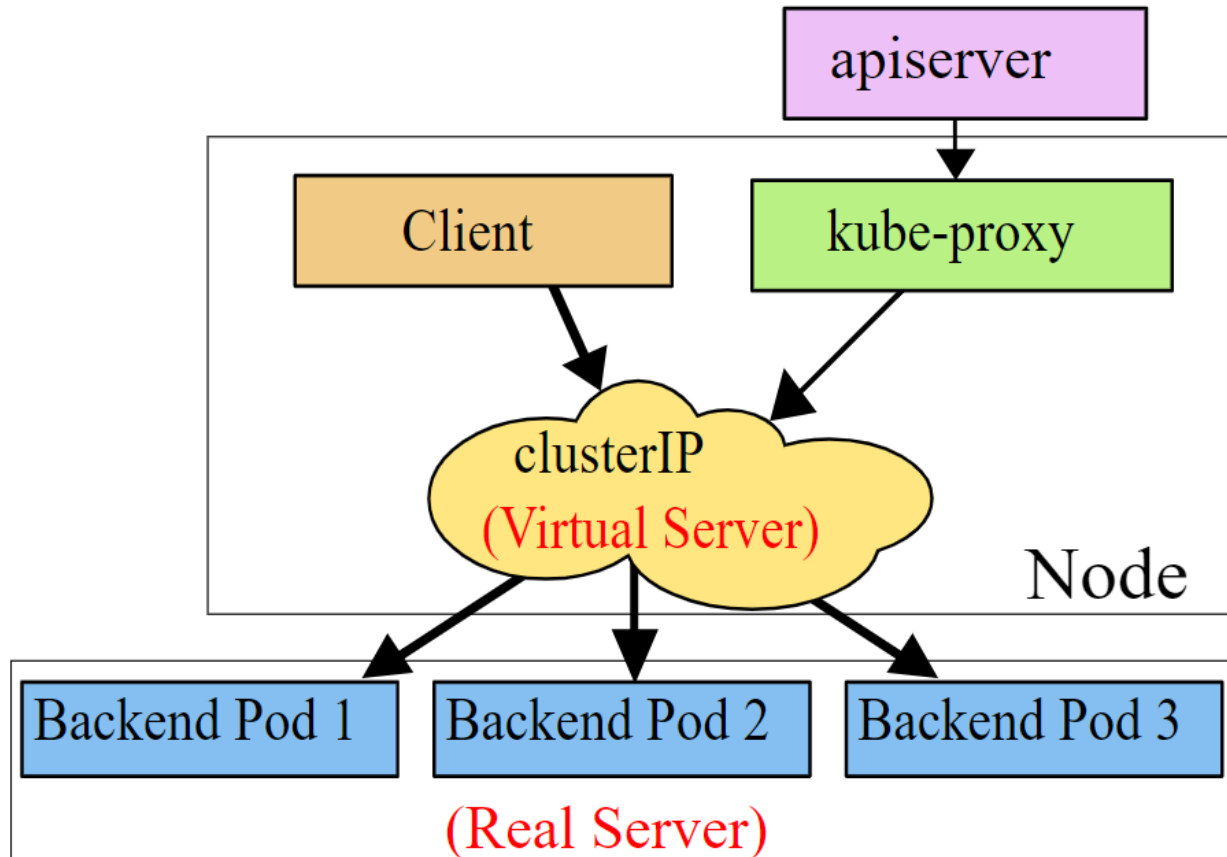
Practical architecture examples:

- ✓ Service Combination Chart
- ✓ Disability isolation
- ✓ Data Consistency
- ✓ Network latency
- ✓ Scalability



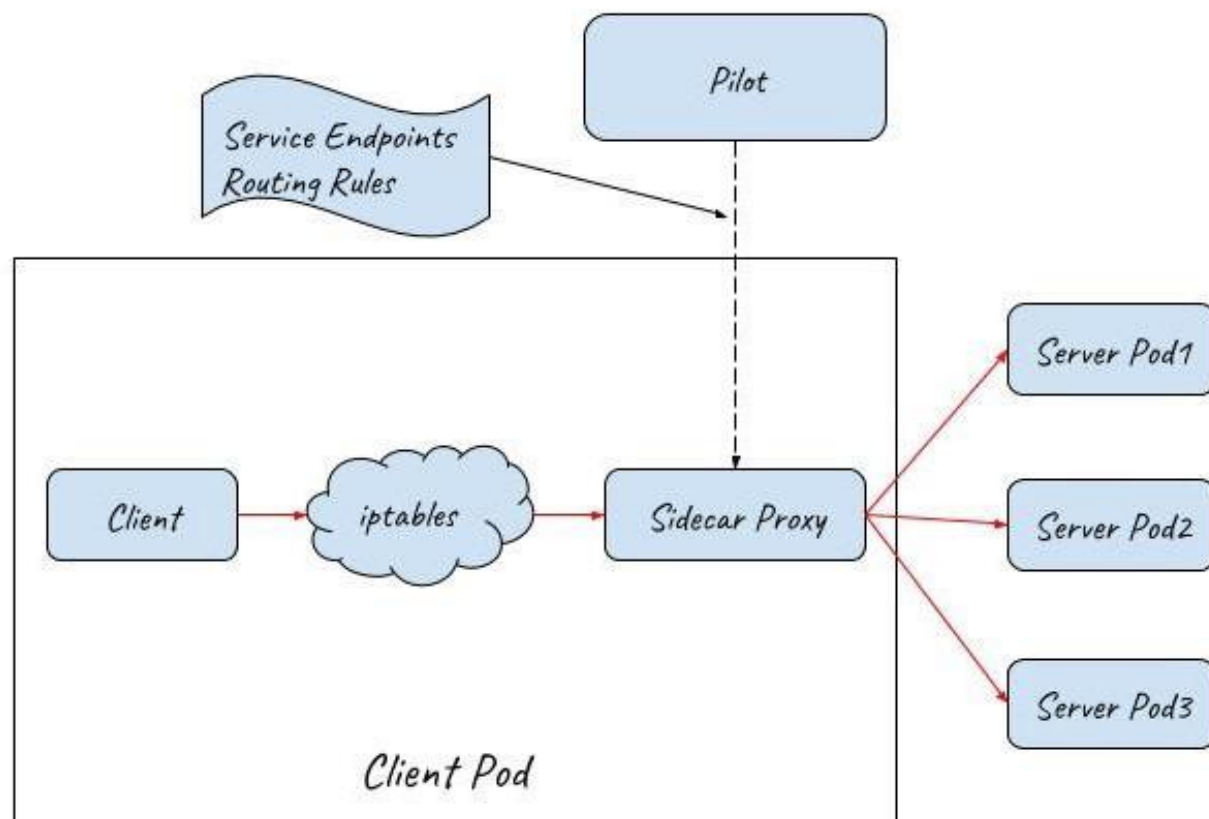
ClusterIP

- ✓ Kubernetes uses Service as an abstraction of Pod groups.
- ✓ Service is bound to ClusterIP, the virtual IP address, so ClusterIP never changes whether the Pod is deleted or restarted



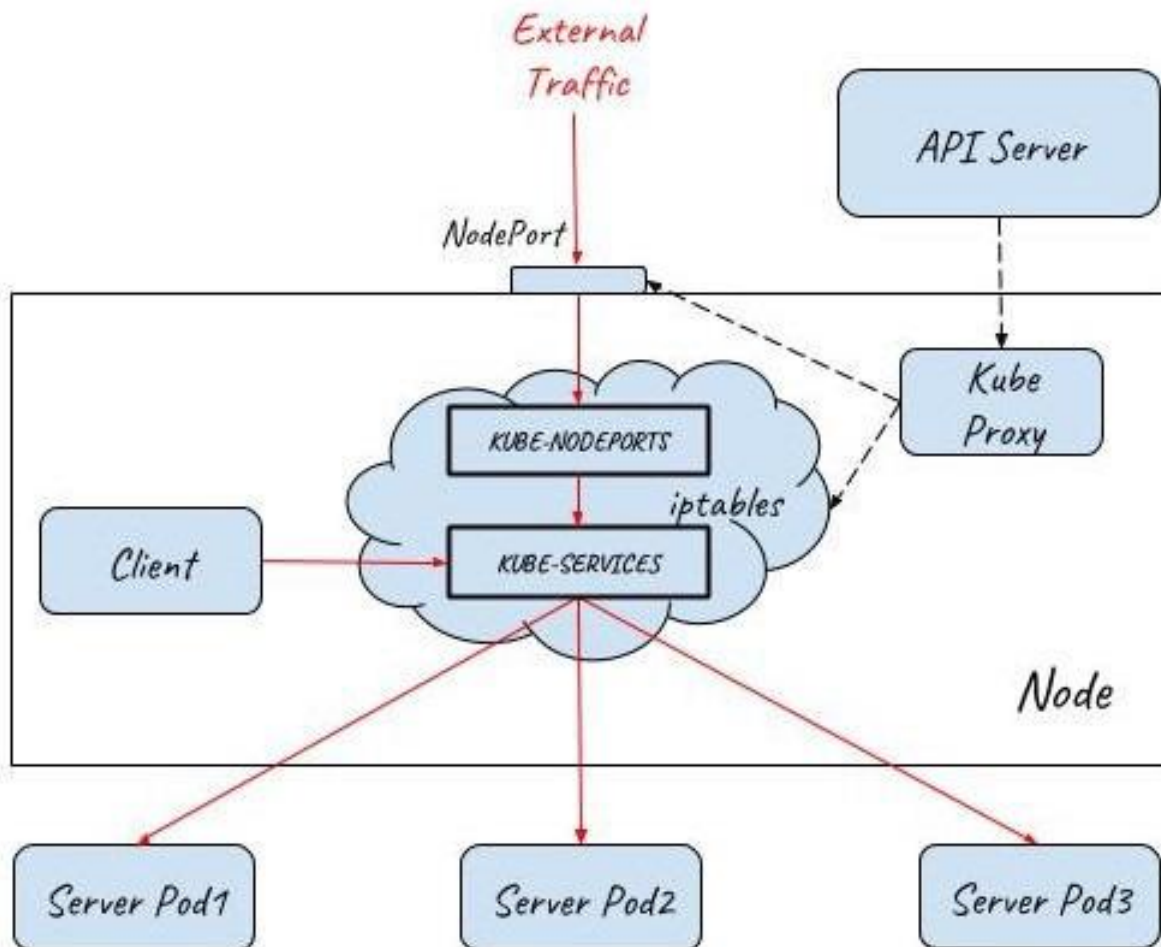
Service mesh

- ✓ Communication between services no longer uses Kube-proxy, but through Istio's Sidecar proxy
- ✓ Client requests come in, redirected by iptables to Sidecar proxy
- ✓ The Sidecar proxy selects the Backend pod and forwards the request according to the service discovery and routing rule information obtained from the (istio) Control plane



NodePort

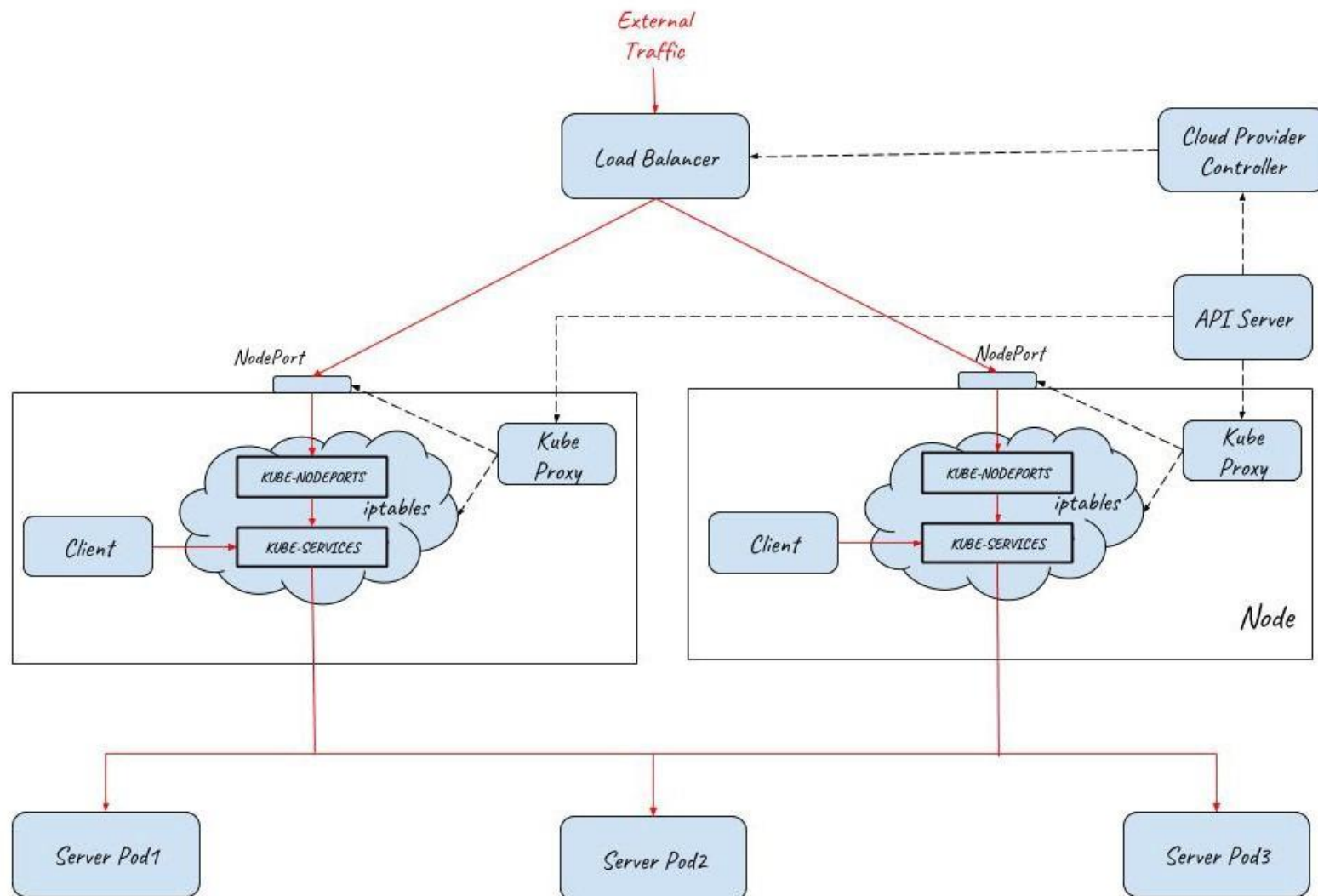
- ✓ Using NodePort, Kubernetes creates a port for service on the host so that it can access the service
- ✓ When a service is declared as a NodePort type, the Kube-proxy creates a port in the Node and waits to receive it from that port
- ✓ Kube-proxy does not directly accept traffic from the node network, but instead creates an iptables rule that takes the traffic sent to the NodePort and redirects it to the Backend Pod



LoadBalancer

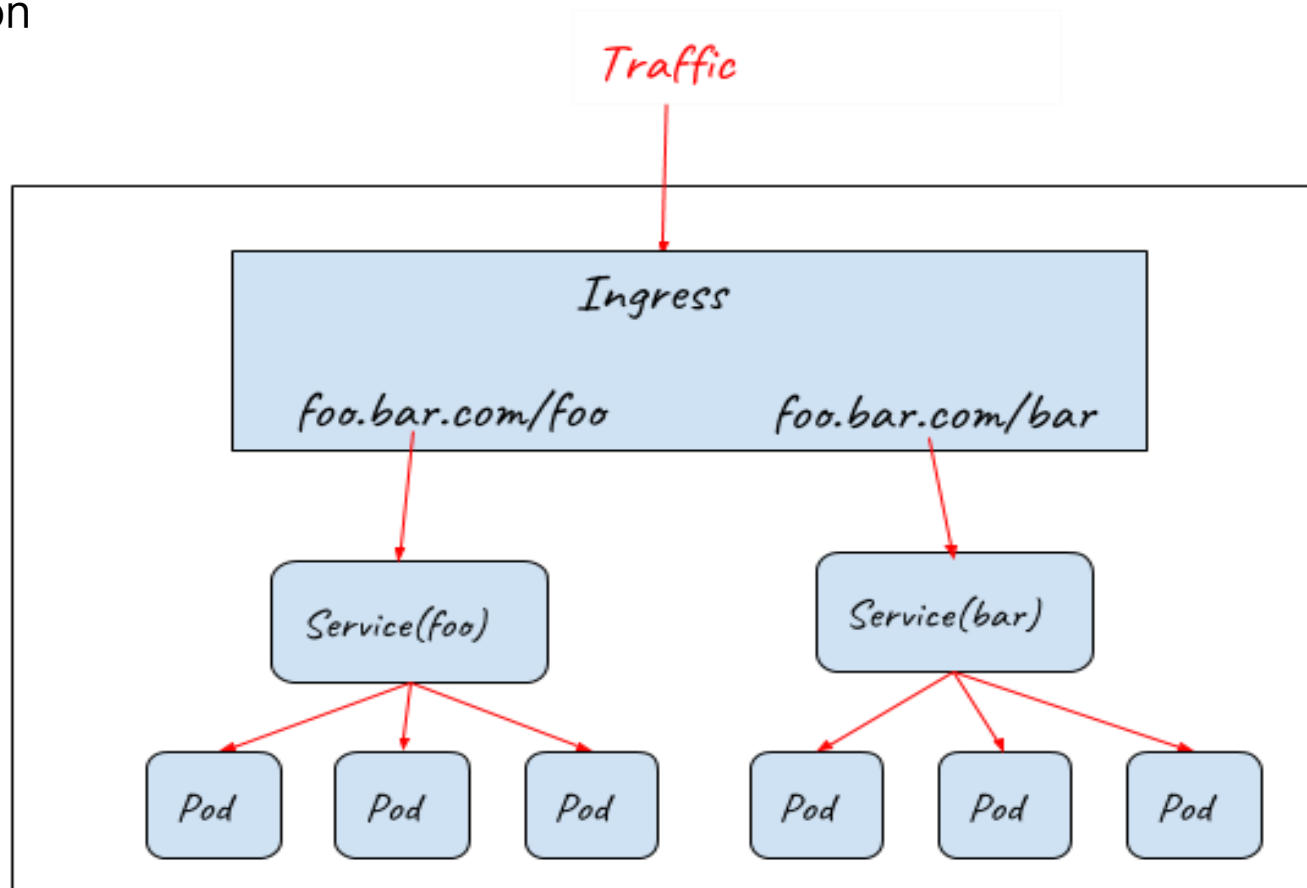
- ✓ Not suitable for NodePortProduction
- ✓ All nodes may crash, and may be removed from the Kubernetes cluster
- ✓ When a new Node is created, Node IP is dynamically assigned in the Address Pool, so Node IP cannot be set to a well-known IP.
- ✓ A single node is a single point of failure (SPOF) on the system
- ✓ A single node becomes a bottleneck for the system
- ✓ You can declare a service as a Loadbalancer type to create an L4 Loadbalancer in front of multiple nodes. This L4 Loadbalancer is outside the network and requires a Cloud Provider Controller for provisioning
- ✓ Kubernetes LoadBalancer operates on OSI 4 Layer (L4), which means that inbound traffic is delivered only to the Backend Service based on IP and Port only

LoadBalancer



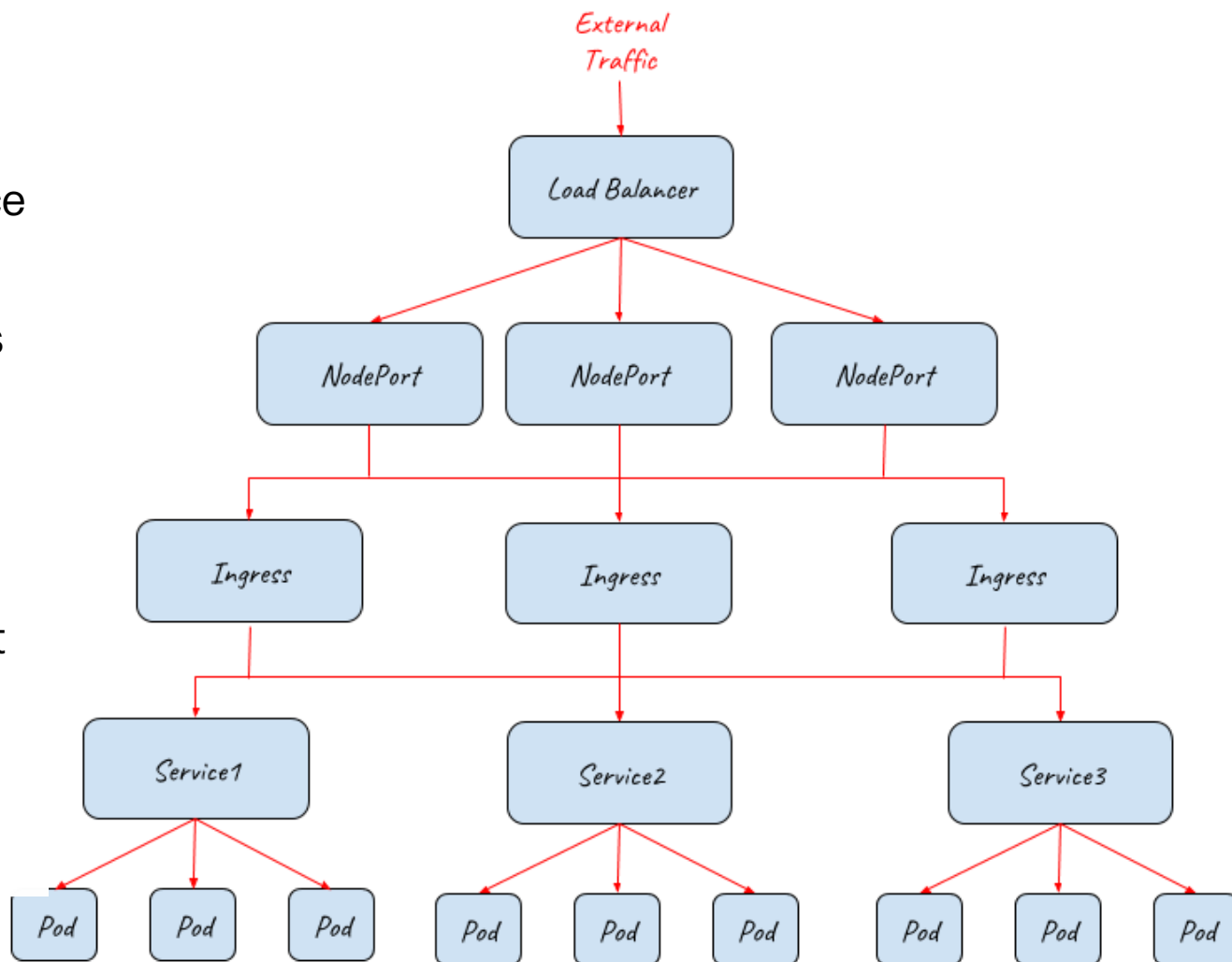
Ingress

- ✓ Ingress resources define only requirements for L7 Load balancer and how to route requests to Backend Service based on HTTP URL/Host, TLS key, and certification configuration



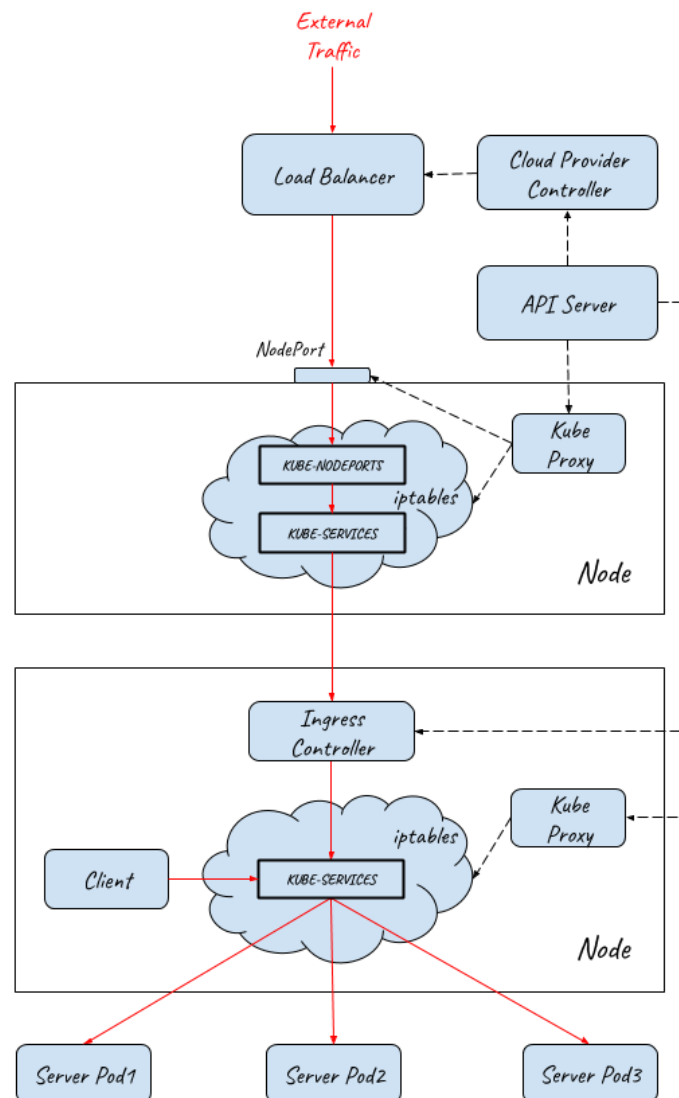
Ingress

- ✓ Ingress Controller provides a unified entrance to HTTP Service in the cluster
- ✓ Ingress Controller itself is also deployed as a Pod inside the cluster and cannot be accessed directly from outside.
- ✓ Ingress Controllers must work with NodePort and LoadBalancer to provide a full path for external traffic into the cluster



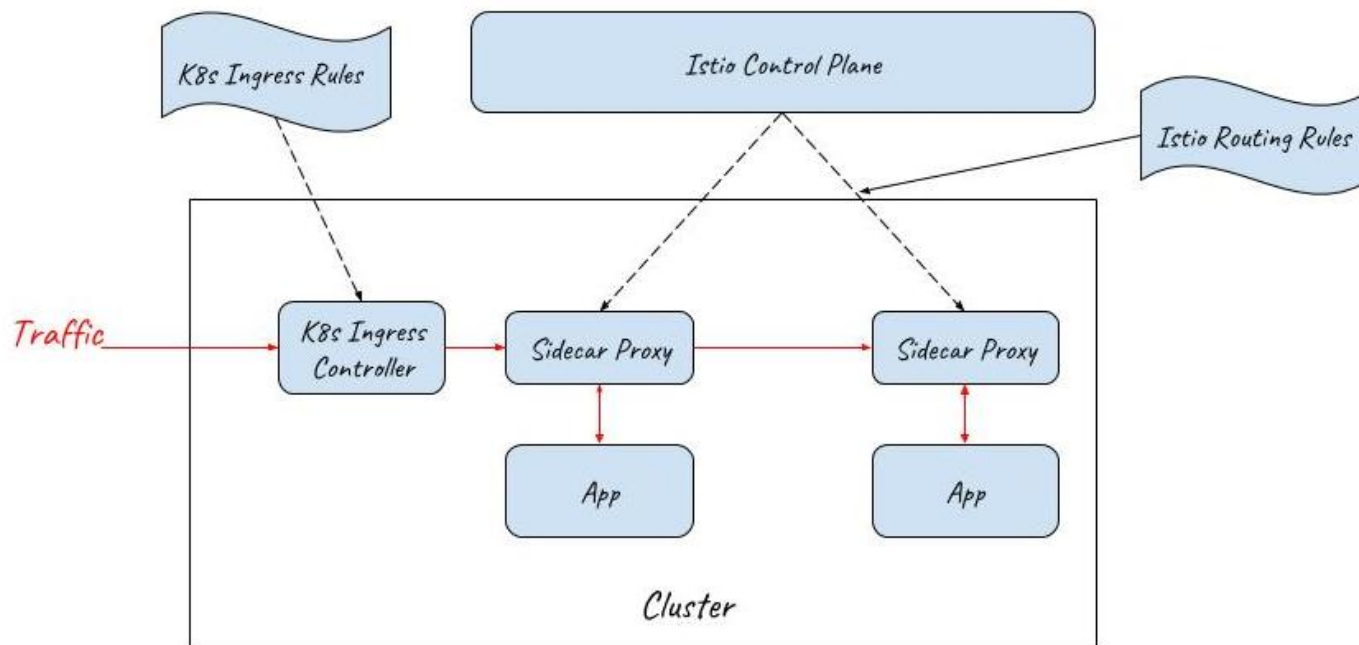
Ingress

- ✓ An input controller is required for input resources to operate in the cluster
- ✓ Input controllers include Nginx, HAProxy, Special Envoy, etc
- ✓ Scale and shrink NodePort, Ingress and Pod tiers appropriately to handle a wide variety of workloads



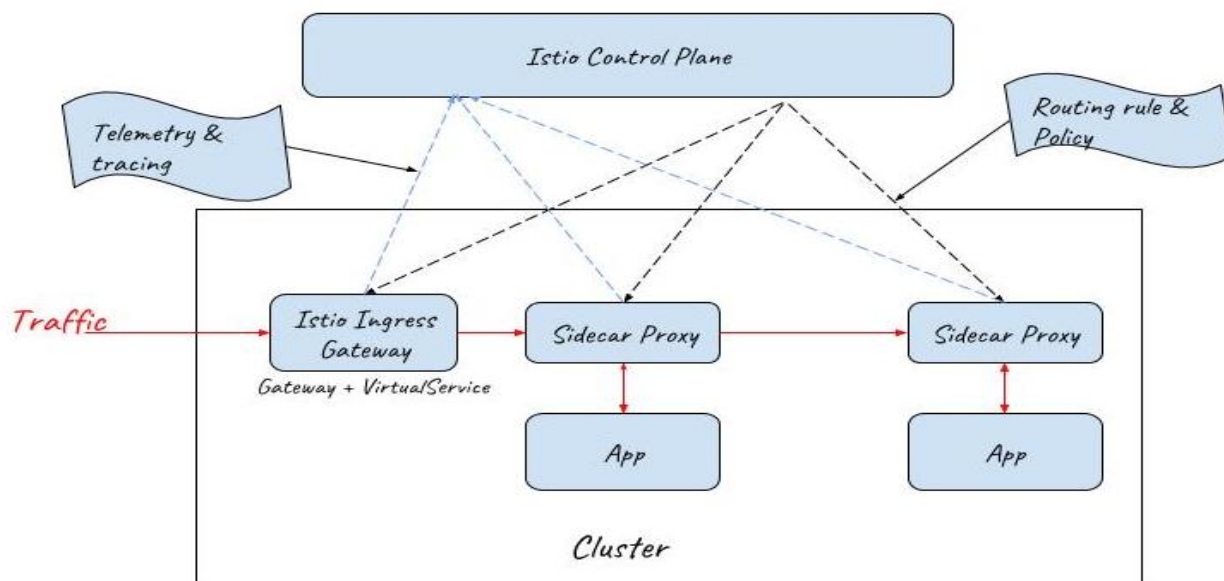
Service Mesh API G/W

- ✓ Ingress cannot be managed by Istio Control Plane
- ✓ Must be created with Kubernetes Ingress rule, does not fit well with Mesh
- ✓ Ingress provides only very basic L7 features
- ✓ Advanced routing rules, distributed tracking, policy checking, metrics collection, etc. are not provided with the same functionality as the Mesh Sidecar Proxy



Istio G/W

- ✓ Istio Gateway resources are much simpler than Kubernetes Ingress
- ✓ Only L4–L6 functions such as Port, Host, TLS key, and authentication can be configured.
- ✓ The gateway is bound to the Istio VirtualService resource, the same resource used for routing configuration within the mesh. This provides the same functions mentioned above inside and at the entrance of the mesh



Features that need to be added in Istio G/W



K8S Ingress

Load balancing
SSL termination
Virtual hosting



Istio Gateway

Load balancing
SSL termination
Virtual hosting
Advanced traffic routing
Fault injection
Other benefits brought
by Istio



API Gateway

Load balancing
SSL termination
Virtual hosting
Traffic routing
API lifecycle management
API Operation Monitoring
API access authorization
API key management
API transformation
API Billing and Rate limiting

API Gateway + Sidecar Proxy

- ✓ API Gateway and Sidecar Proxy Use as Input Gateway for Service Mesh
- ✓ The API gateway already has L7 capabilities, so the Sidecar proxy only needs to provide routing capabilities for Istio VirtualService resources

